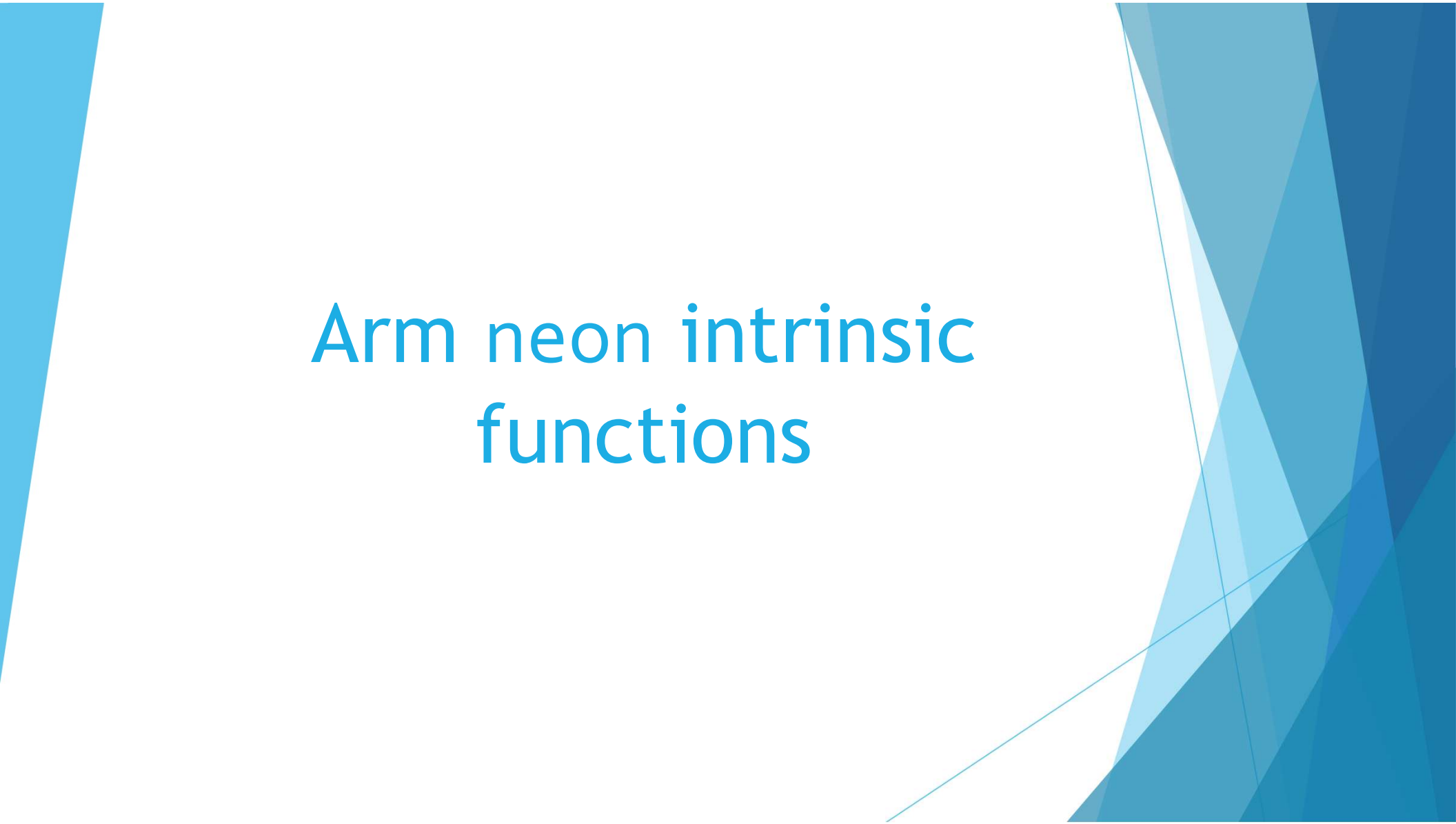
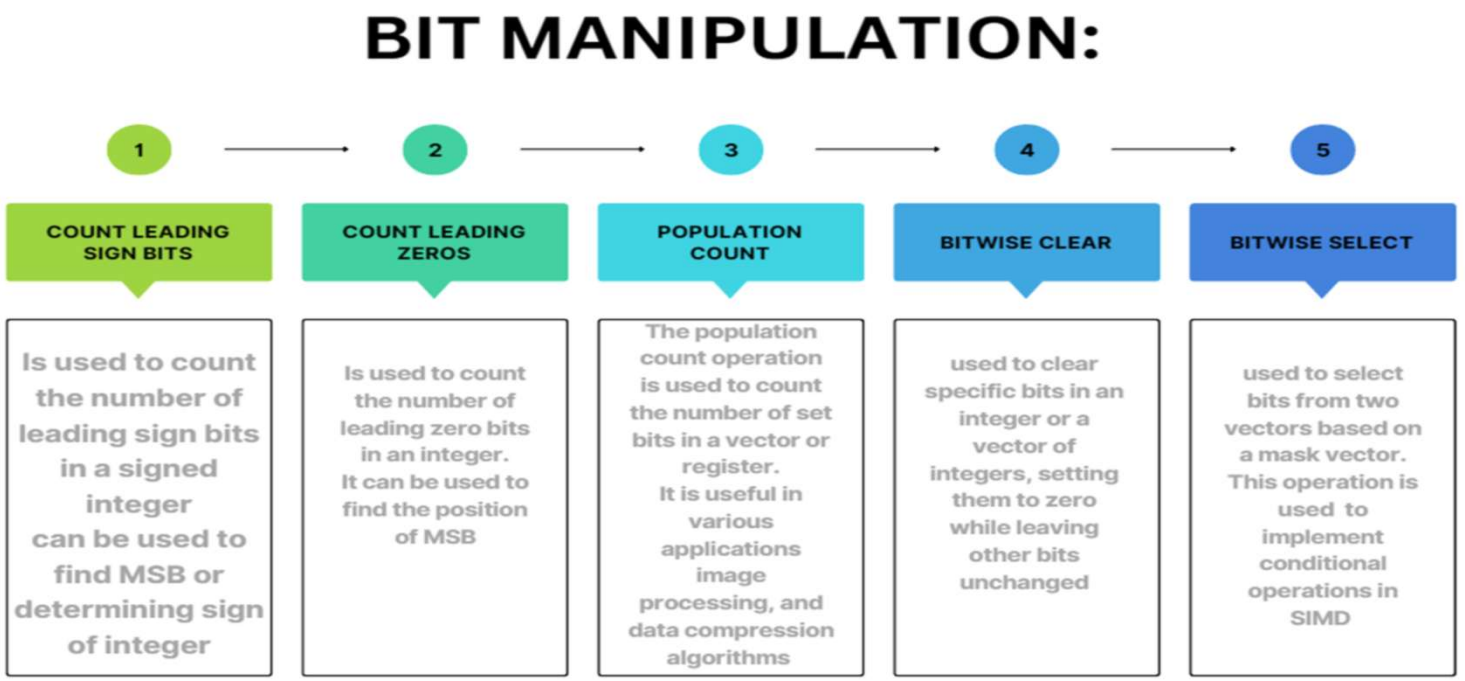
# Arm neon intrinsic functions



## Bit manipulation operation in Neon:

- The bit manipulation operations in ARM NEON manipulates individual bits within each element simultaneously across the entire vector.
- NEON processor can perform the bit manipulation operations on multiple data elements simultaneously, which leads to significant speedup in the performance as compared to iterating over individual bits in a loop.



# Compare operations

- The Compare operations are used to perform element wise comparison between elements of two vectors and generate a another vector based on the comparison result. This  vector can then be used for various conditional operations.
- The different types of compare operations:

- Bitwise equal
- Bitwise equal to zero
- Greater than or equal to
- Less than or equal to
- Greater than
- Less than
- Absolute greater than or equal to
- Absolute less than or equal to
- Absolute greater than
- Absolute less than
- Bitwise not equal to zero
- Greater than zero
- Less than or equal to zero
- Equal to

# Complex Arithmetic operations:

- Complex arithmetic operations enable efficient processing of complex numbers in SIMD. These operations can be useful in digital signal processing and scientific computing where complex numbers are used.
- Different instructions in complex arithmetic operations:

1. **Complex Addition**: This instruction is used to add two complex numbers efficiently. This instruction adds the real part and the imaginary parts of the two vectors separately.
2. **Complex multiply-Accumulate:** This instruction combines complex multiplication and complex addition into a single instruction. It is useful in digital signal processing where complex arithmetic operations are involved.
3. **Complex multiply-Accumulate by scalar:** This instructions involves multiplying each element of a complex vector by a scalar value and accumulating the results.

# Load operations

- Load operations enables loading of data from memory to NEON registers for further processing. This operation enables  efficient processing of large datasets in simultaneously.
- Different load instructions:

1. Stride: This instruction controls how data is accessed and processed. This instruction can control how data is fetched or stored in memory, enabling efficient data processing and manipulation.
2. Load: This instruction loads an element from memory, and writes the result as a scalar to the SIMD&FP register.

## Store operations:

- This store operations involves storing the contents of NEON registers into memory. The store operation is essential for completing the data processing pipeline in many NEON algorithms as the results of NEON registers are written back to memory for further processing and output.
- The different instructions of store operations:

1. **Stride:** Let's consider an example where you have a vector of 32-bit integers and you want to store it into memory with a stride of 2. This means that you want to skip every other memory location when storing the elements. The stride parameter allows you to specify this skipping behavior.
2. **Store:** This instruction stores a single SIMD&FP register to memory.

## Shift operations:

- This shift operation refers to the process of shifting the bits of a vector (or scalar) left or right by a certain number of positions. Shifting bits to the left is equivalent to multiplying by a power of 2, while shifting bits to the right is equivalent to dividing by a power of 2. This operation is useful for data manipulation and can be used in signal processing applications.
- The different instructions of shift operations:

1. Left shift: This instruction is used to shift the bits of elements within a vector to the left by a specified number of positions. This operation effectively multiplies the elements of the vector by powers of 2.
2. Right shift: This instruction is used to shift the bits of each element in a vector or register to the right by a specified number of positions. Right shifting effectively divides the value by a power of two

# Logical Operations:

These instructions read each vector element from the source SIMD&FP register, performs the bitwise logical operations between the two vectors, and writes the vector to the destination SIMD&FP register.

Types of logical functions:

- **Bitwise NOT (vector).** This instruction reads each vector element from the source SIMD&FP register, places the inverse of each value into a vector, and writes the vector to the destination SIMD&FP register.
- **Bitwise AND (vector).** This instruction performs a bitwise AND between the two source SIMD&FP registers, and writes the result to the destination SIMD&FP register.
- **Bitwise inclusive OR (vector, register).** This instruction performs a bitwise OR between the two source SIMD&FP registers, and writes the result to the destination SIMD&FP register.
- **Bitwise Exclusive OR (vector).** This instruction performs a bitwise Exclusive OR operation between the two source SIMD&FP registers, and places the result in the destination SIMD&FP register.
- **Bitwise Exclusive OR (vector).** This instruction performs a bitwise Exclusive OR operation between the two source SIMD&FP registers, and places the result in the destination SIMD&FP register.

# Datatype Conversions:

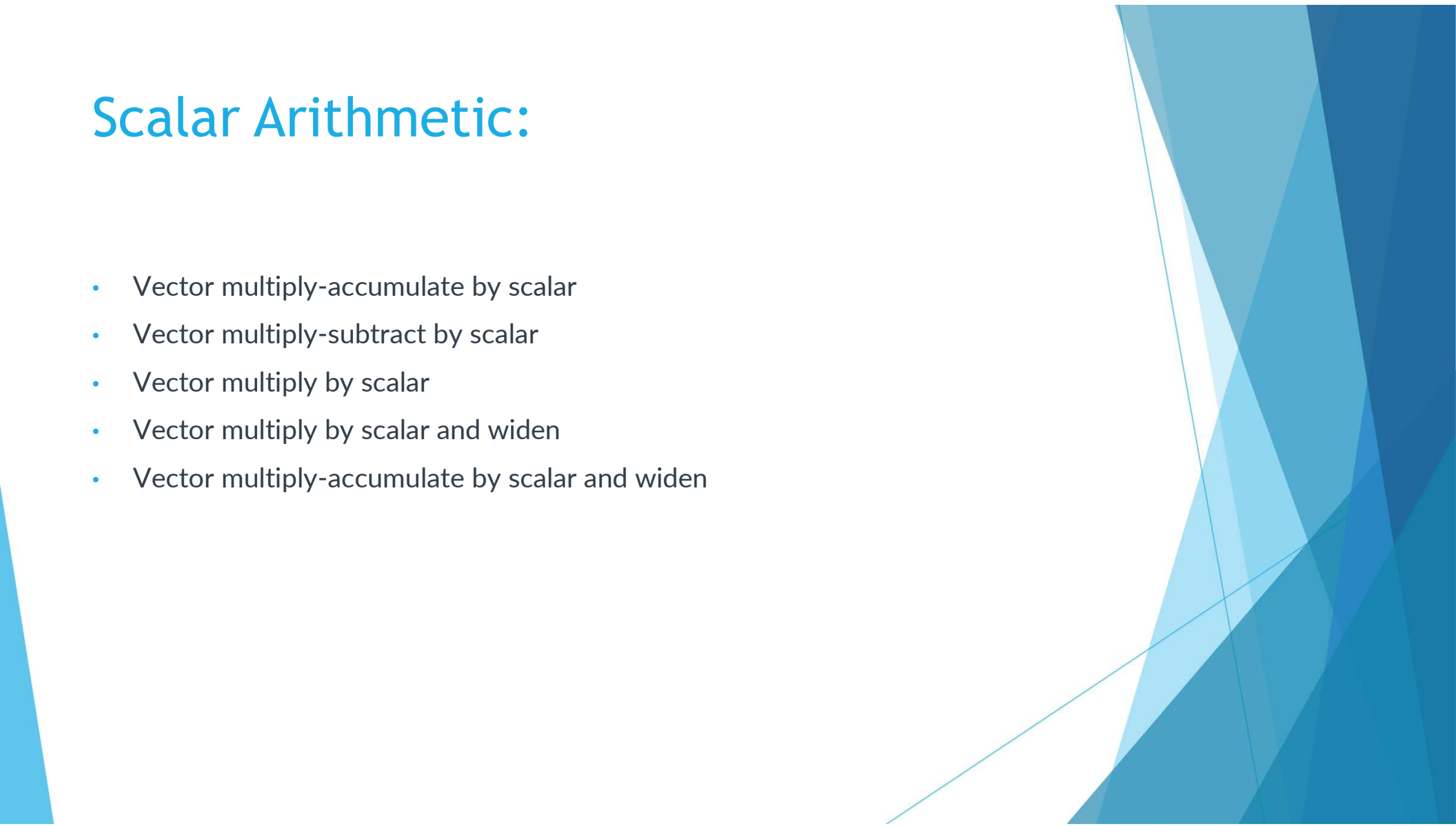
- **Floating-point Convert to Unsigned fixed-point, rounding toward Zero (vector):** This instruction converts a scalar or each element in a vector from floating-point to fixed-point unsigned integer using the Round towards Zero rounding mode, and writes the result to the general-purpose destination register.
- **Reinterpret Casts:** Vector reinterpret cast operations.

# Move Operations:

- **Extract Narrow:** This instruction reads each vector element from the source SIMD&FP register, narrows each value to half the original width, places the result into a vector, and writes the vector to the lower or upper half of the destination SIMD&FP register. The destination vector elements are half as long as the source vector elements.
- **Unsigned saturating extract Narrow:** This instruction reads each vector element from the source SIMD&FP register, saturates each value to half the original width, places the result into a vector, and writes the vector to the destination SIMD&FP register. All the values in this instruction are unsigned integer values.
- **Wide:** This instruction reads each vector element from the source SIMD&FP register and writes the vector to the destination SIMD&FP register directly and writes the vector to the destination SIMD&FP register.

# Scalar Arithmetic:

- Vector multiply-accumulate by scalar
- Vector multiply-subtract by scalar
- Vector multiply by scalar
- Vector multiply by scalar and widen
- Vector multiply-accumulate by scalar and widen



# Vector Arithmetic:

- **Add (vector).** This instruction adds corresponding elements in the two source SIMD&FP registers, places the results into a vector, and writes the vector to the destination SIMD&FP register.
- **Unsigned Add Long (vector).** This instruction adds each vector element in the lower or upper half of the first source SIMD&FP register to the corresponding vector element of the second source SIMD&FP register, places the result into a vector, and writes the vector to the destination SIMD&FP register. The destination vector elements are twice as long as the source vector elements. All the values in this instruction are unsigned integer values.
- **Unsigned Halving Add.** This instruction adds corresponding unsigned integer values from the two source SIMD&FP registers, shifts each result right one bit, places the results into a vector, and writes the vector to the destination SIMD&FP register.
- **Unsigned saturating Add.** This instruction adds the values of corresponding elements of the two source SIMD&FP registers, places the results into a vector, and writes the vector to the destination SIMD&FP register.

- **Multiply (vector).** This instruction multiplies corresponding elements in the vectors of the two source SIMD&FP registers, places the results in a vector, and writes the vector to the destination SIMD&FP register.
- **Multiply-Add to accumulator (vector).** This instruction multiplies corresponding elements in the vectors of the two source SIMD&FP registers, and accumulates the results with the vector elements of the destination SIMD&FP register.
- **Unsigned Multiply-Add Long (vector).** This instruction multiplies the vector elements in the lower or upper half of the first source SIMD&FP register by the corresponding vector elements of the second source SIMD&FP register, and accumulates the results with the vector elements of the destination SIMD&FP register. The destination vector elements are twice as long as the elements that are multiplied.
- **Multiply-Subtract from accumulator (vector).** This instruction multiplies corresponding elements in the vectors of the two source SIMD&FP registers, and subtracts the results from the vector elements of the destination SIMD&FP register.
- **Unsigned Multiply-Subtract Long (vector).** This instruction multiplies corresponding vector elements in the lower or upper half of the two source SIMD&FP registers, and subtracts the results from the vector elements of the destination SIMD&FP register. The destination vector elements are twice as long as the elements that are multiplied. All the values in this instruction are unsigned integer values.

- **Subtract (vector).** This instruction subtracts each vector element in the second source SIMD&FP register from the corresponding vector element in the first source SIMD&FP register, places the result into a vector, and writes the vector to the destination SIMD&FP register.
- **Subtract (vector).** This instruction subtracts each vector element in the second source SIMD&FP register from the corresponding vector element in the first source SIMD&FP register, places the result into a vector, and writes the vector to the destination SIMD&FP register.
- **Unsigned Halving Subtract.** This instruction subtracts the vector elements in the second source SIMD&FP register from the corresponding vector elements in the first source SIMD&FP register, shifts each result right one bit, places each result into a vector, and writes the vector to the destination SIMD&FP register.
- **Unsigned saturating Subtract.** This instruction subtracts the element values of the second source SIMD&FP register from the corresponding element values of the first source SIMD&FP register, places the results into a vector, and writes the vector to the destination SIMD&FP register.
- **Subtract returning High Narrow.** This instruction subtracts each vector element in the second source SIMD&FP register from the corresponding vector element in the first source SIMD&FP register, places the most significant half of the result into a vector, and writes the vector to the lower or upper half of the destination SIMD&FP register. All the values in this instruction are signed integer values.

- **Unsigned Absolute Difference (vector).** This instruction subtracts the elements of the vector of the second source SIMD&FP register from the corresponding elements of the first source SIMD&FP register, places the absolute values of the results into a vector, and writes the vector to the destination SIMD&FP register.

1. **Unsigned Maximum (vector).** This instruction compares corresponding elements in the vectors in the two source SIMD&FP registers, places the larger of each pair of unsigned integer values into a vector, and writes the vector to the destination SIMD&FP register.
2. **Unsigned Minimum (vector).** This instruction compares corresponding vector elements in the two source SIMD&FP registers, places the smaller of each of the two unsigned integer values into a vector, and writes the vector to the destination SIMD&FP register.

- **Unsigned Reciprocal Estimate.** This instruction reads each vector element from the source SIMD&FP register, calculates an approximate inverse for the unsigned integer value, places the result into a vector, and writes the vector to the destination SIMD&FP register.
- **Add Pairwise (vector).** This instruction creates a vector by concatenating the vector elements of the first source SIMD&FP register after the vector elements of the second source SIMD&FP register, reads each pair of adjacent vector elements from the concatenated vector, adds each pair of values together, places the result into a vector, and writes the vector to the destination SIMD&FP register.

# Vector Manipulation:

- **Insert vector element from another vector element:** This instruction copies the vector element of the source SIMD&FP register to the specified vector element of the destination SIMD&FP register.
- **Duplicate vector element to vector or scalar:** This instruction duplicates the vector element at the specified element index in the source SIMD&FP register into a scalar or each element in a vector, and writes the result to the destination SIMD&FP register.
- **Combine vectors:** Join two smaller vectors into a single larger vector.
- **Split vectors:** Duplicate vector element to vector or scalar. This instruction duplicates the vector element at the specified element index in the source SIMD&FP register into a scalar or each element in a vector, and writes the result to the destination SIMD&FP register.

- **Extract vector from pair of vectors.** This instruction extracts the lowest vector elements from the second source SIMD&FP register and the highest vector elements from the first source SIMD&FP register, concatenates the results into a vector, and writes the vector to the destination SIMD&FP register vector. The index value specifies the lowest vector element to extract from the first source register, and consecutive elements are extracted from the first, then second, source registers until the destination vector is filled.
- **Reverse elements in 64-bit doublewords (vector).** This instruction reverses the order of 8-bit, 16-bit, or 32-bit elements in each doubleword of the vector in the source SIMD&FP register, places the results into a vector, and writes the vector to the destination SIMD&FP register.
- **Reverse elements in 64-bit doublewords (vector).** This instruction reverses the order of 8-bit, 16-bit, or 32-bit elements in each doubleword of the vector in the source SIMD&FP register, places the results into a vector, and writes the vector to the destination SIMD&FP register.
- **Set vector Lane:** Move vector element to another vector element.

- **Transpose Elements**
- **Zip vectors (secondary).** This instruction reads adjacent vector elements from the upper half of two source SIMD&FP registers as pairs, interleaves the pairs and places them into a vector, and writes the vector to the destination SIMD&FP register. The first pair from the first source register is placed into the two lowest vector elements, with subsequent pairs taken alternately from each source register.
- **Unzip vectors (secondary).** This instruction reads corresponding odd-numbered vector elements from the two source SIMD&FP registers, places the result from the first source register into consecutive elements in the lower half of a vector, and the result from the second source register into consecutive elements in the upper half of a vector, and writes the vector to the destination SIMD&FP register.